Programação Estruturada e Tipos de Dados

CTeSP em Redes e Sistemas Informáticos

Docente: Mário Amorim

Ano Letivo: 2025/2026

Objetivos da Unidade Curricular

- Compreender os conceitos fundamentais da programação estruturada
- Dominar a sintaxe básica da linguagem Python e suas estruturas de controlo
- Conhecer e aplicar os diferentes tipos de dados e estruturas disponíveis
- Desenvolver algoritmos para resolver problemas simples e complexos
- Adquirir boas práticas de programação e modularização de código

Resultados de Aprendizagem Esperados

- Capacidade de analisar problemas e desenvolver soluções algorítmicas
- Implementação de programas em Python utilizando estruturas de controlo adequadas
- Manipulação eficiente de diferentes tipos de dados e estruturas
- Criação de programas modulares com funções reutilizáveis
- Aplicação de boas práticas de programação e documentação de código

O que é programar? (1/2)

Algoritmo

Sequência finita e ordenada de passos bem definidos que resolvem um problema específico.

Programa

Conjunto de instruções que dizem ao computador o que fazer, escritas numa linguagem que o computador entende.

Linguagem de Programação

Método padronizado para comunicar instruções ao computador, com regras sintáticas e semânticas específicas.

```python
# Exemplo de algoritmo em Python
# Calcular a média de três números

def calcular_media(a, b, c):
    soma = a + b + c
    media = soma / 3
    return media

resultado = calcular_media(10, 15, 20)
print(f"A média é: {resultado}")

# Saída: A média é: 15.0
```

O que é programar? (2/2)

Paradigmas de Programação: Diferentes abordagens para resolver problemas através de código.

Programação Estruturada

Foco na organização do código em estruturas lógicas sequenciais.

- Sequência, seleção e iteração
- Funções e procedimentos
- Fluxo de controlo previsível
- Fácil de aprender e entender

Programação Orientada a Objetos

Organiza o código em objetos que contêm dados e comportamentos.

- Classes e objetos
- Encapsulamento e herança
- Polimorfismo
- Reutilização de código

Nesta unidade curricular, focaremos no paradigma de ***Programação Estruturada*** *utilizando Python.*

Porquê Python?

Sintaxe Simples e Legível

Código limpo e fácil de entender, ideal para iniciantes.

Multiplataforma

Funciona em Windows, macOS, Linux e outros sistemas operativos.

Usado no Mundo Real

Amplamente utilizado em ciência de dados, IA, desenvolvimento web, automação e muito mais.

Vasto Ecossistema

Milhares de bibliotecas e frameworks disponíveis para diversas aplicações.

Comunidade Ativa

Grande comunidade de desenvolvedores, documentação extensa e suporte online.

"Python é uma linguagem que enfatiza a legibilidade do código e permite expressar conceitos em menos linhas de código do que seria possível em linguagens como C++ ou Java."

Ambiente de Desenvolvimento (1/2)

1 Instalar Python

Descarregar e instalar a versão mais recente do Python em python.org

Importante: Marcar a opção "Add Python to PATH" durante a instalação

2 Instalar Visual Studio Code

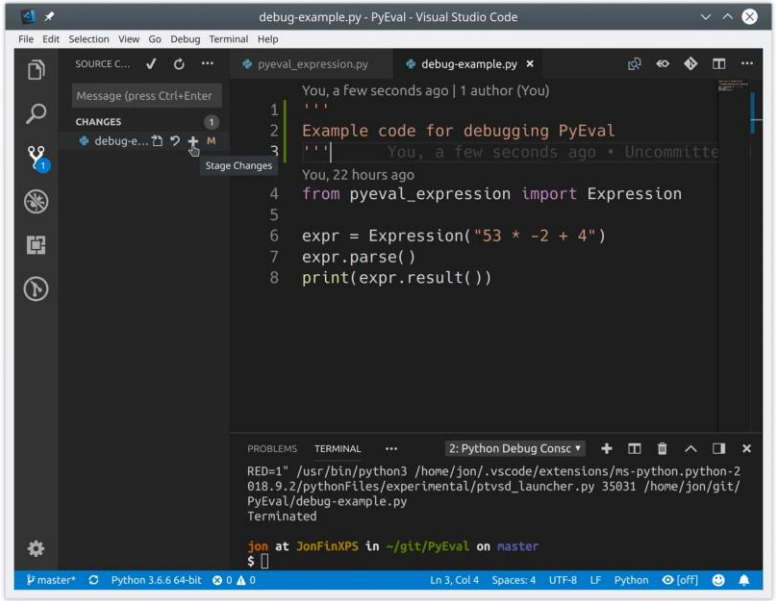
Descarregar e instalar o VS Code em code.visualstudio.com

3 Instalar a Extensão Python

No VS Code, abrir a secção de extensões (Ctrl+Shift+X) e procurar por "Python"

4 Criar e Executar o Primeiro Programa

Criar um ficheiro .py e executar com o botão "Run" ou com F5



Visual Studio Code com a extensão Python

Ambiente de Desenvolvimento (2/2)

Alternativas ao VS Code: Outras opções para programar em Python

Thonny

IDE Python para iniciantes, com interface simples e ferramentas de depuração visual.

- Instalação simples
- Depurador visual
- Ideal para iniciantes
- Gratuito

Replit

IDE online que permite programar diretamente no navegador sem instalação.

- Baseado na web
- Colaboração em tempo real
- Múltiplas linguagens
- Versão gratuita disponível

Google Colab

Ambiente baseado em Jupyter Notebooks com acesso a GPUs gratuitas.

- Notebooks interativos
- GPUs gratuitas
- Integração com Google Drive
- Ideal para ciência de dados

IDLE

IDE básico incluído com a instalação padrão do Python.

- Já incluído com Python
- Interface simples
- Depurador integrado
- Sem configuração adicional

Variáveis e Tipos de Dados (1/2)

O que são variáveis?

Variáveis são espaços na memória que armazenam valores. Em Python, não é necessário declarar o tipo de uma variável.

Sintaxe: nome_variavel = valor

int (inteiro)

Números inteiros sem parte decimal.

Exemplos: 42, -7, 0, 1000

float (ponto flutuante)

Números com parte decimal.

Exemplos: 3.14, -0.001, 2.0, 1e6

str (string)

Sequências de caracteres (texto).

Exemplos: "Olá", 'Python', """Texto longo"""

bool (booleano)

Valores lógicos verdadeiro ou falso.

Exemplos: True, False

```
# Exemplos de variáveis e tipos

idade = 25
altura = 1.75
nome = "Ana Silva"
estudante_ativo = True

print(type(idade))
# Saída: <class 'int'>
print(type(altura))
# Saída: <class 'float'>
print(type(nome))
# Saída: <class 'str'>
print(type(estudante_ativo))
# Saída: <class 'bool'>
```

Variáveis e Tipos de Dados (2/2)

Tipagem Dinâmica

Em Python, as variáveis são dinamicamente tipadas, o que significa que o tipo de uma variável é determinado em tempo de execução e pode mudar durante o programa.

```
x = 10
print(type(x))
# Saída: <class 'int'>

x = "Python"
print(type(x))
# Saída: <class 'str'>
```

Conversão de Tipos

Python permite converter valores entre diferentes tipos de dados usando funções de conversão.

Função	Descrição	Exemplo
int()	Converte para inteiro	int("42") → 42
float()	Converte para decimal	float("3.14") → 3.14
str()	Converte para string	str(42) → "42"
bool()	Converte para booleano	bool(1) → True

Regras para Nomes de Variáveis

- Devem começar com uma letra ou underscore (_)
- Podem conter letras, números e underscores
- Não podem começar com números
- Não podem ser palavras reservadas (if, for, while, etc.)
- São sensíveis a maiúsculas/minúsculas (case-sensitive)

```
# Exemplos de conversão de tipos

# Entrada de dados (sempre retorna string)
entrada = input("Digite um número: ")
# Suponha que o utilizador digitou "25"

# Conversão para inteiro
numero = int(entrada)
resultado = numero * 2
print(f"O dobro é: {resultado}")
# Saída: O dobro é: 50

# Conversão para string
idade = 30
mensagem = "Tenho " + str(idade) + " anos"
print(mensagem)
# Saída: Tenho 30 anos
```

Operadores

Operadores Aritméticos

Operador	Descrição	Exemplo
+	Adição	5 + 3 = 8
-	Subtração	5 - 3 = 2
*	Multiplicação	5 * 3 = 15
/	Divisão	5 / 3 = 1.6667
//	Divisão inteira	5 // 3 = 1
%	Módulo (resto)	5 % 3 = 2
**	Exponenciação	5 ** 3 = 125

Operadores Lógicos

Operador	Descrição	Exemplo
and	E lógico	True and False → False
or	OU lógico	True or False → True
not	NÃO lógico	not True → False

Operadores Relacionais

Operador	Descrição	Exemplo
==	Igual a	5 == 5 → True
!=	Diferente de	5 != 3 → True
>	Maior que	5 > 3 → True
<	Menor que	5 < 3 → False
>=	Maior ou igual	5 >= 5 → True
<=	Menor ou igual	5 <= 3 → False

Exemplos Práticos

```
a = 10   b = 5
# Operadores aritméticos
soma = a + b
print(f"Soma: {soma}")
# Operadores relacionais
maior = a > b
print(f"A é maior que B? {maior}")
# Operadores lógicos
resultado = (a > 5) and (b < 10)
print(f"Resultado: {resultado}")
```

Entrada e Saída de Dados

input()

Recebe dados do utilizador através do teclado. Retorna sempre uma string.

```
input(prompt)
```

Exemplo:

```
nome = input("Digite seu nome: ")
print("Olá, " + nome + "!")
```

```
Digite seu nome: Ana
Olá, Ana!
```

print()

Exibe informações na tela.

```
print(objeto1, objeto2, ..., sep=' ', end='\n')
```

Parâmetros:

- `sep` : separador entre os objetos (padrão: espaço)
- `end` : caractere no final da impressão (padrão: nova linha)

f-strings

Forma moderna e legível de formatar strings em Python (a partir da versão 3.6).

```
# Exemplo completo de entrada e saída

# Entrada de dados
nome = input("Digita o teu nome: ")
idade_str = input("Digita a tua idade: ")
idade = int(idade_str)

# Processamento
ano_nascimento = 2025 - idade

# Saída de dados
print(f"Olá, {nome}!")
print(f“Tu nasceste em {ano_nascimento}.")

print("Daqui a 10 anos terás", idade + 10, "anos.")
```

```
Digita o teu nome: Maria
Digita a tua idade: 30
Olá, Maria!
Tu nasceste em 1994.
Daqui a 10 anos terás 40 anos.
```

Exercício 1 (prático)

Calculadora Básica

Crie um programa que solicite dois números ao utilizador e mostre a soma, subtração, multiplicação e divisão desses números.

Requisitos:

1. Solicitar dois números ao utilizador
2. Converter as entradas para números
3. Calcular soma, subtração, multiplicação e divisão
4. Verificar se o segundo número não é zero antes de dividir
5. Mostrar os resultados formatados

```
Digite o primeiro número: 10
Digite o segundo número: 5
Soma: 15.0
Subtração: 5.0
Multiplicação: 50.0
Divisão: 2.0
```

```python
# Calculadora básica
# Entrada de dados
num1_str = input("Digite o primeiro número: ")
num2_str = input("Digite o segundo número: ")

# Conversão para float
num1 = float(num1_str)
num2 = float(num2_str)

# Cálculos
soma = num1 + num2
subtracao = num1 - num2
multiplicacao = num1 * num2
# Saída dos resultados
print(f"Soma: {soma}")
print(f"Subtração: {subtracao}")
print(f"Multiplicação: {multiplicacao}")
# Verificação para divisão
if num2 != 0:
divisao = num1 / num2
print(f"Divisão: {divisao}")
else:
print("Não é possível dividir por
zero!")
```

Estruturas de Controlo – Condicionais (1/2)

if

Executa um bloco de código apenas se a condição for verdadeira.

```
if condição:
# bloco de código
```

if-else

Executa um bloco se a condição for verdadeira, ou outro bloco se for falsa.

```
if condição:
# bloco de código se verdadeiro
else:
# bloco de código se falso
```

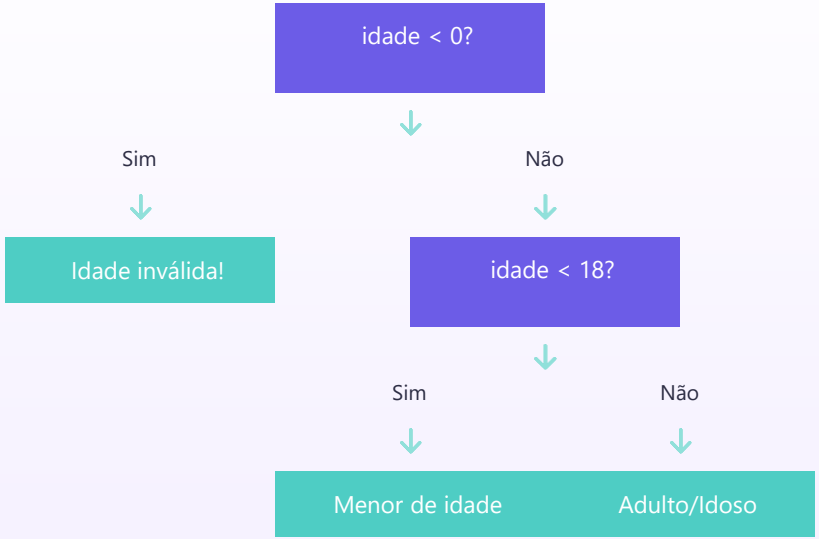
if-elif-else

Verifica múltiplas condições em sequência.

```
if condição1:
# bloco de código se condição1 for verdadeira
elif condição2:
# bloco de código se condição2 for verdadeira
else:
# bloco de código se todas as condições forem falsas
```

Estruturas de Controlo – Condicionais (2/2)

```
# Exemplo de estruturas condicionais

idade = int(input("Digite sua idade: "))

if idade < 0:
print("Idade inválida!")
elif idade < 18:
print("Você é menor de idade.")
anos_faltantes = 18 - idade
print(f"Faltam {anos_faltantes} anos para a
maioridade.")
elif idade < 65:
print("Você é adulto.")
else:
print("Você é idoso.")

# Operador ternário
status = "Par" if idade % 2 == 0 else "Ímpar"
print(f"Sua idade é um número {status}.")
```



Estruturas de Controlo - Ciclos

for

Executa um bloco de código para cada item em uma sequência (lista, string, etc.).

```
for variável in sequência:
# bloco de código
```

```
# Exemplo de ciclo for
for i in range(5):
print(i, end=" ")
```

```
0 1 2 3 4
```

Nota: A função `range(n)` gera uma sequência de números de 0 até n-1.

while

Executa um bloco de código enquanto uma condição for verdadeira.

```
while condição:
# bloco de código
```

```
# Exemplo de ciclo while
contador = 0
while contador < 5:
print(contador, end=" ")
contador += 1
```

```
0 1 2 3 4
```

Estruturas de Controlo - Ciclos

break e continue

break : Interrompe a execução do ciclo.
continue : Pula para a próxima iteração do ciclo.

```
# Exemplo de break
for i in range(10):
if i == 5:
break
print(i, end=" ")
```

```
0 1 2 3 4
```

```
# Exemplo de continue
for i in range(10):
if i % 2 == 0:
continue
print(i, end=" ")
```

```
1 3 5 7 9
```

Exemplo Prático

```
# Soma dos números de 1 a 10
soma = 0
for numero in range(1, 11):
soma += numero
print(f"A soma dos números de 1 a 10 é:
{soma}")
# Verificação de número primo
num = int(input("Digite um número: "))
eh_primo = True
if num <= 1:
eh_primo = False
else:
for i in range(2, int(num**0.5) + 1):
if num % i == 0:
eh_primo = False
break
if eh_primo:
print(f"{num} é um número primo.")
else:
print(f"{num} não é um número primo.")
```

Exercício 2 (prático)

Comparação de Números

Crie um programa que solicite dois números ao utilizador e indique qual é o maior.

Requisitos:

1. Solicitar dois números ao utilizador
2. Converter as entradas para números
3. Comparar os números e determinar qual é o maior
4. Considerar também o caso em que os números são iguais
5. Mostrar o resultado ao utilizador

```python
# Comparação de números

# Entrada de dados
num1_str = input("Digite o primeiro número: ")
num2_str = input("Digite o segundo número: ")

# Conversão para float
num1 = float(num1_str)
num2 = float(num2_str)

# Comparação e saída
if num1 > num2:
print(f"O primeiro número ({num1}) é maior que o segundo ({num2}).")
elif num2 > num1:
print(f"O segundo número ({num2}) é maior que o primeiro ({num1}).")
else:
print(f"Os números são iguais ({num1}).")
```

Exercício 3 (prático)

Classificação de Número

Crie um programa que solicite um número ao utilizador e indique se é positivo, negativo ou zero.

Requisitos:

1. Solicitar um número ao utilizador
2. Converter a entrada para número
3. Verificar se o número é positivo (maior que zero)
4. Verificar se o número é negativo (menor que zero)
5. Verificar se o número é zero
6. Mostrar a classificação ao utilizador

Dica:
Este exercício pode ser resolvido com apenas duas condições (if-else), mas a solução com três condições (if-elif-else) é mais clara e explícita.

```python
# Classificação de número

# Entrada de dados
numero_str = input("Digite um número: ")

# Conversão para float
numero = float(numero_str)

# Verificação e classificação
if numero > 0:
print(f"O número {numero} é positivo.")
elif numero < 0:
print(f"O número {numero} é negativo.")
else:
print("O número é zero.")
```

```
Digite um número: -5
O número -5.0 é negativo.
```

Ciclo for (1/2)

O que é o ciclo for?

O ciclo `for`

é utilizado para iterar sobre uma sequência (lista, tupla, dicionário, conjunto ou string) ou outros objetos iteráveis.

```
for variável in sequência:
# bloco de código a ser executado
```

Função range()

A função `range()`

gera uma sequência de números que pode ser usada com o ciclo

```
range(stop)  # 0 até stop-1
range(start, stop)  # start até stop-1
range(start, stop, step)  # start até stop-1
com passo step
```

```
# Exemplos de range()
print(list(range(5)))
print(list(range(2, 8)))
print(list(range(1, 10, 2)))
```

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5, 6, 7]
[1, 3, 5, 7, 9]
```

Ciclo for (2/2)

Exemplos de ciclo for

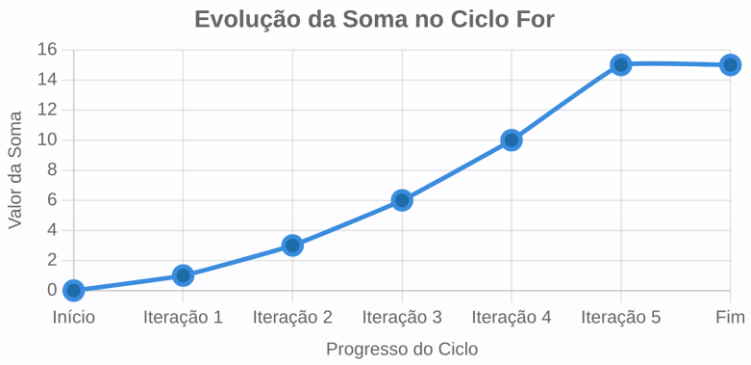
```
# Iteração sobre uma lista
frutas = ["maçã", "banana", "laranja",
"uva"]
for fruta in frutas:
print(f"Eu gosto de {fruta}")
```

```
Eu gosto de maçã
Eu gosto de banana
Eu gosto de laranja
Eu gosto de uva
```

```
# Iteração com range()
soma = 0
for i in range(1, 6):
soma += i
print(f"Iteração {i}: soma = {soma}")
```

```
Iteração 1: soma = 1
Iteração 2: soma = 3
Iteração 3: soma = 6
Iteração 4: soma = 10
Iteração 5: soma = 15
```

Visualização do ciclo for



Nota: O ciclo for é ideal quando se sabe antecipadamente o número de iterações a realizar ou quando se precisa iterar sobre elementos de uma coleção.

Ciclo while (1/2)

O que é o ciclo while?

O ciclo `while` executa um bloco de código enquanto uma condição especificada for verdadeira. É usado quando não sabemos antecipadamente quantas iterações serão necessárias.

```
while condição:
# bloco de código a ser executado
# deve incluir algo que eventualmente altere
a condição
```

Ciclos Infinitos

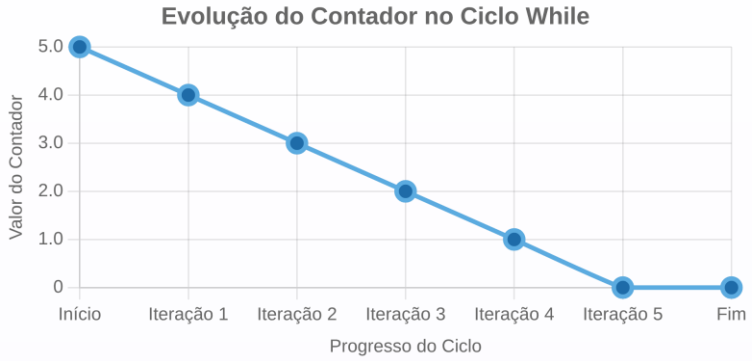
Se a condição do ciclo `while` nunca se tornar falsa, o ciclo continuará executando indefinidamente, criando um ciclo infinito.

```
# Exemplo de ciclo infinito (NÃO EXECUTE!)
contador = 1
while contador > 0:
print(f"Iteração {contador}")
contador += 1  # contador sempre
aumenta, nunca será <= 0
```

Atenção! Sempre garanta que a condição do ciclo `while` eventualmente se torne falsa, ou use `break` para sair do ciclo.

Ciclo while (2/2)

Visualização do ciclo while



Nota: O ciclo while

é ideal para situações em que o número de iterações não é conhecido antecipadamente, como validação de entrada do utilizador ou processamento até que uma condição seja atendida.

Exemplos de ciclo while

```
# Contagem regressiva
contador = 5
while contador > 0:
print(f"Contagem: {contador}")
contador -= 1
print("Lançamento!")
```

```
Contagem: 5
Contagem: 4
Contagem: 3
Contagem: 2
Contagem: 1
Lançamento!
```

```
# Validação de entrada
senha_correta = "python123"
tentativa = ""
while tentativa != senha_correta:
tentativa = input("Digite a senha: ")
if tentativa != senha_correta:

print("Senha incorreta. Tente novamente.")

print("Senha correta! Acesso permitido.")
```

Diferenças entre for e while

Ciclo for

- Usado quando o número de iterações é conhecido antecipadamente
- Itera sobre sequências (listas, strings, etc.) ou range()
- Mais conciso e menos propenso a erros
- Variável de controle é gerida automaticamente

```
# Exemplo de ciclo for
for i in range(1, 6):
print(i, end=" ")
# Saída: 1 2 3 4 5
```

Ciclo while

- Usado quando o número de iterações não é conhecido
- Executa enquanto uma condição for verdadeira
- Mais flexível para condições complexas
- Requer gestão manual da variável de controle

```
# Exemplo de ciclo while
i = 1
while i <= 5:
print(i, end=" ")
i += 1
# Saída: 1 2 3 4 5
```

Exercício 4 (prático) (1/2)

Cálculo de Média

Crie um programa que leia 5 números introduzidos pelo utilizador e calcule a média desses números.

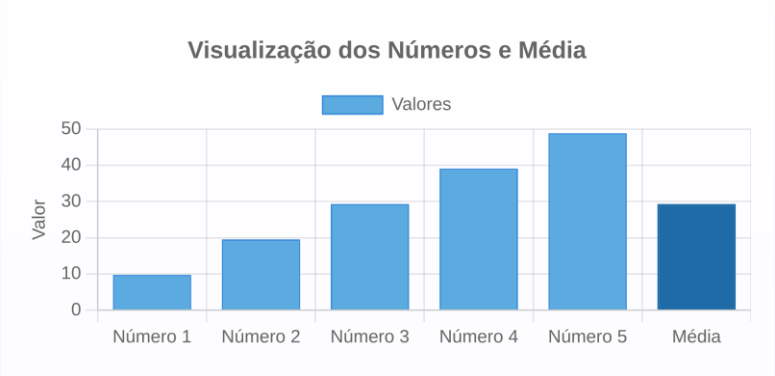
Requisitos:

1. Utilizar um ciclo for para ler 5 números
2. Converter cada entrada para número
3. Acumular a soma dos números
4. Calcular a média (soma dividida por 5)
5. Mostrar a média ao utilizador

```python
# Cálculo de média de 5 números

# Inicialização da variável para acumular a soma
soma = 0

# Ciclo para ler 5 números
for i in range(5):
numero_str = input(f"Digite o {i+1}º número: ")
numero = float(numero_str)
soma += numero
print(f"Soma atual: {soma}")

# Cálculo da média
media = soma / 5

# Apresentação do resultado
print(f"A média dos 5 números é: {media:.2f}")
```

Exercício 4 (prático) (2/2)

```
Digite o 1º número: 10
Soma atual: 10.0
Digite o 2º número: 20
Soma atual: 30.0
Digite o 3º número: 30
Soma atual: 60.0
Digite o 4º número: 40
Soma atual: 100.0
Digite o 5º número: 50
Soma atual: 150.0
A média dos 5 números é: 30.00
```



Exercício 5 (prático) (1/2)

Validação de Entrada

Crie um programa que continue a pedir um número ao utilizador até que este introduza um número positivo.

Requisitos:

1. Utilizar um ciclo while para repetir a leitura
2. Solicitar um número ao utilizador
3. Converter a entrada para número
4. Verificar se o número é positivo (maior que zero)
5. Se não for positivo, mostrar mensagem e repetir
6. Se for positivo, mostrar mensagem e terminar

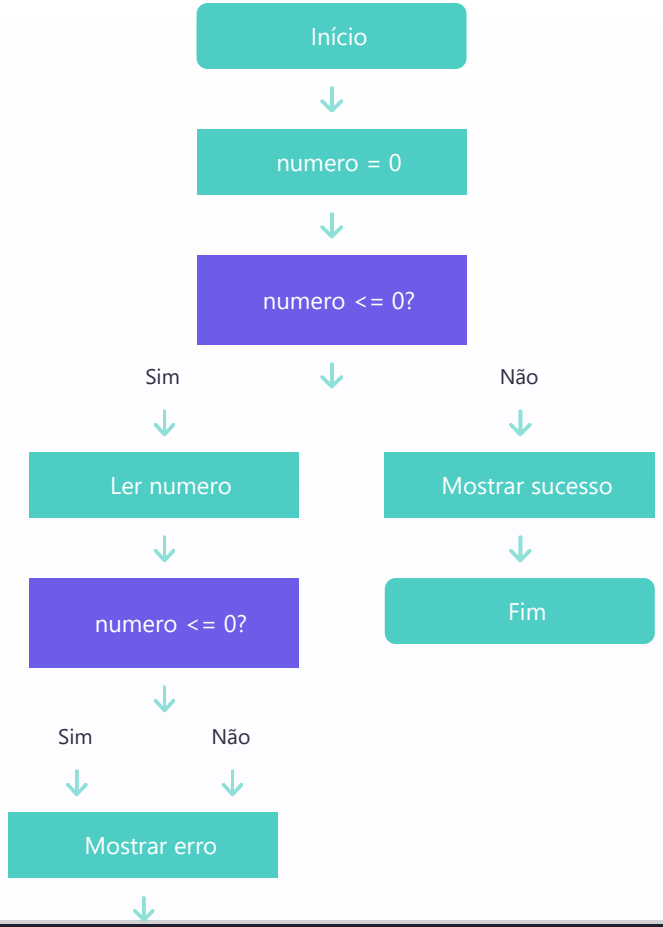
```python
# Validação de entrada - número positivo

# Inicialização da variável
numero = 0

# Ciclo while para validação
while numero <= 0:
numero_str = input("Digite um número positivo: ")
numero = float(numero_str)
if numero <= 0:
print("Erro: O número deve ser positivo. Tente novamente.")


# Mensagem final
print(f"Obrigado! Você digitou o número positivo {numero}.")
```

```
Digite um número positivo: -5
Erro: O número deve ser positivo. Tente novamente.
Digite um número positivo: 0
Erro: O número deve ser positivo. Tente novamente.
Digite um número positivo: 10
Obrigado! Você digitou o número positivo 10.0.
```

Exercício 5 (prático) (2/2)

Nota: Este exercício demonstra um caso típico onde o ciclo while é mais adequado que o for pois não sabemos quantas iterações serão necessárias até o utilizador introduzir um valor válido.

Fluxograma do Ciclo While para Validação



Listas (1/4)

O que são Listas?

Listas são coleções ordenadas e mutáveis de elementos. Podem conter elementos de diferentes tipos, incluindo outras listas.

```
# Criação de uma lista
nome_lista = [elemento1, elemento2, elemento3, ...]
```

Criação de Listas

```
# Lista vazia
lista_vazia = []

# Lista de números
numeros = [1, 2, 3, 4, 5]

# Lista de strings
frutas = ["maçã", "banana", "laranja"]

# Lista com diferentes tipos
misturada = [1, "olá", 3.14, True]

# Lista de listas (matriz)
matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Listas (2/4)

Acesso aos Elementos

Os elementos de uma lista são acessados por índices, que começam em 0.

```
frutas = ["maçã", "banana", "laranja",
"uva", "pera"]

# Acesso por índice positivo (do início)
print(frutas[0])  # Primeiro elemento
print(frutas[2])  # Terceiro elemento

# Acesso por índice negativo (do fim)
print(frutas[-1])  # Último elemento
print(frutas[-2])  # Penúltimo elemento
```

```
maçã
laranja
pera
uva
```

Visualização dos Índices:

maçã	banana	laranja	uva	pera
0 -5	1 -4	2 -3	3 -2	4 -1

Função len()

A função `len()` retorna o número de elementos em uma lista.

```
frutas = ["maçã", "banana", "laranja",
"uva", "pera"]
tamanho = len(frutas)
print(f"A lista tem {tamanho} elementos.")
```

```
A lista tem 5 elementos.
```

Nota: O último índice válido de uma lista é sempre `len(lista) - 1`, pois os índices começam em 0.

Listas (3/4)

Métodos de Listas

Python oferece vários métodos para manipular listas:

Método	Descrição
append()	Adiciona um elemento ao final da lista
insert()	Insere um elemento em uma posição específica
remove()	Remove a primeira ocorrência de um valor
pop()	Remove e retorna um elemento por índice
sort()	Ordena a lista (in-place)
reverse()	Inverte a ordem dos elementos

Operador in

O operador `in` verifica se um elemento está presente na lista.

```
frutas = ["maçã", "banana", "laranja",
"uva"]

if "banana" in frutas:
print("Banana está na lista!")

if "pera" not in frutas:
print("Pera não está na lista!")
```

```
Banana está na lista!
Pera não está na lista!
```

Listas (4/4)

Adicionar Elementos

```
frutas = ["maçã", "banana"]
print(f"Lista original: {frutas}")
# Adicionar ao final
frutas.append("laranja")
print(f"Após append: {frutas}")
# Inserir em posição específica
frutas.insert(1, "morango")
print(f"Após insert: {frutas}")
```

```
Lista original: ['maçã', 'banana']
Após append: ['maçã', 'banana', 'laranja']
Após insert: ['maçã', 'morango', 'banana', 'laranja']
```

Visualização do append():

Visualização do insert(1, "morango"):

Remover Elementos

```
frutas = ["maçã", "morango", "banana",
"laranja", "uva"]
print(f"Lista original: {frutas}")
# Remover por valor
frutas.remove("morango")
print(f"Após remove: {frutas}")
# Remover por índice
fruta_removida = frutas.pop(2)
print(f"Removido: {fruta_removida}")
print(f"Após pop: {frutas}")
```

```
Lista original: ['maçã', 'morango', 'banana',
'laranja', 'uva']
Após remove: ['maçã', 'banana', 'laranja', 'uva']
Removido: laranja
Após pop: ['maçã', 'banana', 'uva']
```

Nota: A diferença entre `remove()` e `pop()` é que `remove()` elimina por valor, enquanto `pop()` elimina por índice e retorna o elemento removido.

Percorrer Listas com for (1/2)

Iteração por Elementos

A forma mais simples e legível de percorrer uma lista é usando o ciclo `for` para iterar diretamente pelos elementos.

```
frutas = ["maçã", "banana", "laranja",
"uva"]

# Iteração pelos elementos
for fruta in frutas:
print(f"Fruta: {fruta}")
```

```
Fruta: maçã
Fruta: banana
Fruta: laranja
Fruta: uva
```

Iteração por Índices

Quando precisamos dos índices durante a iteração, podemos usar a função `range()` com `len()`.

```
frutas = ["maçã", "banana", "laranja",
"uva"]

# Iteração pelos índices
for i in range(len(frutas)):
print(f"Índice {i}: {frutas[i]}")
```

```
Índice 0: maçã
Índice 1: banana
Índice 2: laranja
Índice 3: uva
```

Percorrer Listas com for (2/2)

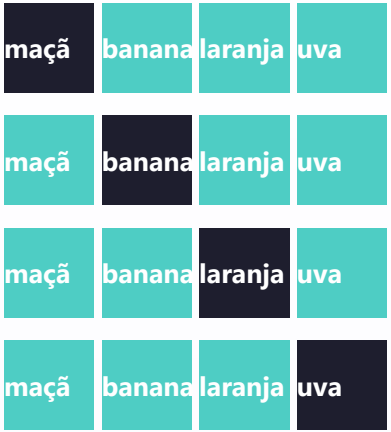
Função enumerate()

A função `enumerate()` permite iterar simultaneamente pelos elementos e seus índices.

```
frutas = ["maçã", "banana", "laranja",
"uva"]

# Iteração com enumerate
for i, fruta in enumerate(frutas):
print(f"Índice {i}: {fruta}")
```

```
Índice 0: maçã
Índice 1: banana
Índice 2: laranja
Índice 3: uva
```

Visualização da Iteração:



Exercício 6 (prático) (1/2)

Lista de Nomes

Crie um programa que armazene os nomes de alguns colegas numa lista e depois imprima cada nome em uma linha separada.

Requisitos:

1. Criar uma lista com pelo menos 5 nomes de colegas
2. Utilizar um ciclo for para percorrer a lista
3. Imprimir cada nome em uma linha separada
4. Adicionar um número de ordem antes de cada nome

Exemplo de Lista de Nomes:

Ana | Bruno | Carlos | Diana | Eduardo

```python
# Lista de nomes dos colegas
# Criação da lista
colegas = ["Ana", "Bruno", "Carlos", "Diana", "Eduardo"]
# Percorrer a lista e imprimir cada nome
print("Lista de colegas:")
for nome in colegas:
print(nome)
# Percorrer a lista com índices
print("\nLista numerada de colegas:")
for i in range(len(colegas)):
print(f"{i+1}. {colegas[i]}")
```

```
Lista de colegas:
Ana
Bruno
Carlos
Diana
Eduardo

Lista numerada de colegas:
1. Ana
2. Bruno
3. Carlos
4. Diana
5. Eduardo
```

Exercício 6 (prático) (2/2)

Lista de Nomes

Crie um programa que armazene os nomes de alguns colegas numa lista e depois imprima cada nome em uma linha separada.

Requisitos:

1. Criar uma lista com pelo menos 5 nomes de colegas
2. Utilizar um ciclo for para percorrer a lista
3. Imprimir cada nome em uma linha separada
4. Adicionar um número de ordem antes de cada nome

Exemplo de Lista de Nomes:

Ana | Bruno | Carlos | Diana | Eduardo

Solução Alternativa com enumerate()

```
# Usando enumerate para obter índice e valor
print("Lista numerada de colegas (com enumerate):")
for i, nome in enumerate(colegas):
print(f"{i+1}. {nome}")
```

```
Lista numerada de colegas (com enumerate):
1. Ana
2. Bruno
3. Carlos
4. Diana
5. Eduardo
```

Funções (1/6)

O que são Funções?

Funções são blocos de código reutilizáveis que realizam uma tarefa específica. Elas permitem organizar o código em unidades lógicas e evitar repetição.

```
def nome_da_funcao():
# bloco de código
# instruções da função

# Chamada da função
nome_da_funcao()
```

Definição de Funções

Em Python, funções são definidas usando a palavra-chave `def`, seguida pelo nome da função e parênteses.

```
# Definição de uma função simples
def saudacao():
print("Olá! Bem-vindo ao curso de Python.")
print("Estamos a aprender sobre funções.")

# Chamada da função
saudacao()
```

```
Olá! Bem-vindo ao curso de Python.
Estamos a aprender sobre funções.
```

Funções (2/6)

Chamada de Funções

Para executar uma função, basta escrever o seu nome seguido de parênteses. Isso é chamado de "chamada de função".

```
# Definição de duas funções
def mostrar_linha():
print("-" * 30)

def mostrar_titulo(texto):
mostrar_linha()
print(f"{texto.center(30)}")
mostrar_linha()

# Chamadas das funções
mostrar_titulo("MENU PRINCIPAL")
mostrar_titulo(“REGISTO DE CLIENTES")
```

```
------------------------------
MENU PRINCIPAL
------------------------------
------------------------------
REDISTO DE CLIENTES
------------------------------
```

Benefícios das Funções

Reutilização de código:
Escreva o código uma vez e use-o várias vezes.

Modularidade:
Divida o programa em partes menores e mais gerenciáveis.

Legibilidade:
Torna o código mais organizado e fácil de entender.

Manutenção:
Facilita a correção de erros e a implementação de melhorias.

Abstração:
Esconde detalhes de implementação, focando no que a função faz, não em como faz.

Fluxo de Execução de uma Função

Chamada da Função: saudacao()

↓

Execução do Código da Função

↓

Retorno ao Ponto de Chamada

Funções (3/6)

Parâmetros

Parâmetros são valores que passamos para uma função quando a chamamos. Eles permitem que a função trabalhe com dados diferentes a cada chamada.

```
def nome_funcao(parametro1, parametro2):
# código que usa parametro1 e
parametro2

# Chamada com argumentos
nome_funcao(valor1, valor2)
```

```
# Função com parâmetros
def saudacao_personalizada(nome, periodo):
if periodo.lower() == "manhã":
print(f"Bom dia, {nome}!")
elif periodo.lower() == "tarde":
print(f"Boa tarde, {nome}!")
else:
print(f"Boa noite, {nome}!")

# Chamadas da função
saudacao_personalizada("Ana", "manhã")
saudacao_personalizada("Carlos", "tarde")
```

```
Bom dia, Ana!
Boa tarde, Carlos!
```

Funções (4/6)

Retorno de Valores

Funções podem retornar valores usando a palavra-chave `return`

Isso permite que a função calcule um resultado e o devolva para ser usado onde a função foi chamada.

```
# Função com retorno
def calcular_area_retangulo(largura, altura):
area = largura * altura
return area

# Chamada e uso do valor retornado
area_sala = calcular_area_retangulo(4, 5)
print(f"A área da sala é {area_sala} metros quadrados.")

# Uso direto do retorno
print(f"A área do quarto é {calcular_area_retangulo(3, 4)} metros quadrados.")
```

```
A área da sala é 20 metros quadrados.
A área do quarto é 12 metros quadrados.
```

Funções (5/6)

Valores por Defeito

Parâmetros podem ter valores por defeito (default), que são usados quando o argumento correspondente não é fornecido na chamada da função.

```
# Função com valores por defeito
def formatar_texto(texto, maiusculas=False, prefixo=""):
if maiusculas:
texto = texto.upper()
return f"{prefixo}{texto}"

# Diferentes formas de chamar
print(formatar_texto("olá mundo"))
print(formatar_texto("olá mundo", True))
print(formatar_texto("olá mundo", prefixo="Mensagem: "))
print(formatar_texto("olá mundo", True, "AVISO: "))
```

```
olá mundo
OLÁ MUNDO
Mensagem: olá mundo
AVISO: OLÁ MUNDO
```

Tipo de Parâmetr	Descrição	Exemplo
Posicional	Passado pela ordem	func(1, 2, 3)
Nomeado	Passado pelo nome	func(a=1, c=3, b=2)
Por defeito	Valor usado se não especificado	def func(a=0, b=1):

Funções (6/6)

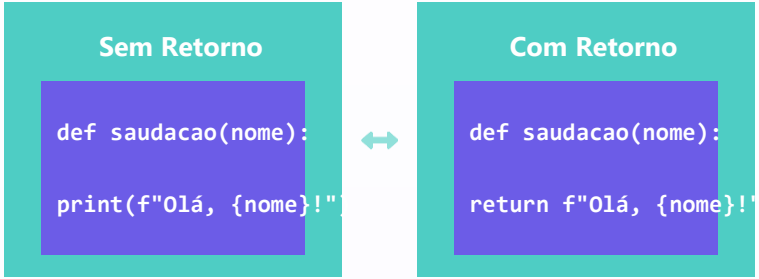
Escopo de Variáveis

Variáveis definidas dentro de uma função são locais a essa função e não podem ser acessadas fora dela. Variáveis definidas fora de qualquer função são globais.

```
# Demonstração de escopo
x = 10  # Variável global

def mostrar_x():

print(f"Valor de x dentro da função: {x}")


def alterar_local():

x = 20  # Variável local, não altera a global

print(f"Valor de x local: {x}")

mostrar_x()
alterar_local()

print(f"Valor de x global após função: {x}")
```

```
Valor de x dentro da função: 10
Valor de x local: 20
Valor de x global após função: 10
```

Comparação: Sem Retorno vs. Com Retorno



Nota:

Funções com retorno são mais flexíveis, pois permitem armazenar, manipular ou exibir o resultado conforme necessário.

Exercício 7 (prático) (1/2)

Função de Soma

Crie uma função que receba dois números como parâmetros, calcule a soma desses números e retorne o resultado.

Requisitos:

1. Definir uma função chamada somar que receba dois parâmetros
2. Calcular a soma dos dois parâmetros
3. Retornar o resultado da soma
4. Testar a função com diferentes pares de números
5. Utilizar o valor retornado em diferentes contextos

Dica:
Este exercício demonstra o conceito básico de funções com parâmetros e retorno de valores. É um bom ponto de partida para entender como as funções podem ser usadas para encapsular operações comuns.

```python
# Definição da função de soma
def somar(a, b):
"""
Soma dois números e retorna o
resultado.
Parâmetros:
a (float): Primeiro número
b (float): Segundo número

Retorna:
float: A soma de a e b
"""
resultado = a + b
return resultado
# Teste da função
print(f"Soma de 5 e 3: {somar(5, 3)}")
print(f"Soma de -2 e 7: {somar(-2, 7)}")
print(f"Soma de 1.5 e 2.5: {somar(1.5,
2.5)}")

# Utilizando o valor retornado
total = somar(10, 20)
print(f"O total é {total}")
print(f"O dobro da soma é {total * 2}")
```

Exercício 7 (prático) (2/2)

```
Soma de 5 e 3: 8
Soma de -2 e 7: 5
Soma de 1.5 e 2.5: 4.0
O total é 30
O dobro da soma é 60
```

Observação:
A documentação da função (docstring) é uma boa prática em Python. Ela descreve o propósito da função, seus parâmetros e o valor de retorno, facilitando o uso e a manutenção do código.

Fluxo de Execução da Função somar()

Chamada: somar(5, 3)

↓

Processamento: a + b = 5 + 3 = 8

↓

Retorno: 8

↓

Uso do valor retornado: print(f"Soma de 5 e 3: {8}")

Exercício 8 (prático) (1/2)

Função para Verificar Número Par

Crie uma função que receba um número como parâmetro e retorne `True` se o número for par ou `False` se for ímpar.

Requisitos:

- Definir uma função chamada `e_par` que receba um número como parâmetro
- Verificar se o número é divisível por 2 (resto da divisão igual a zero)
- Retornar `True` se for par ou `False` se for ímpar
- Testar a função com diferentes números
- Utilizar o valor retornado em estruturas condicionais

Lembrete:

Um número é par se for divisível por 2, ou seja, se o resto da divisão por 2 for igual a zero. Em Python, o operador `%` (módulo) retorna o resto da divisão.

```python
# Definição da função para verificar se um número é par
def e_par(numero):
"""
Verifica se um número é par.

Parâmetros:
numero (int): O número a ser verificado

Retorna:
bool: True se o número for par, False se for ímpar

"""
return numero % 2 == 0
# Teste da função
print(f"4 é par? {e_par(4)}")
print(f"7 é par? {e_par(7)}")
print(f"0 é par? {e_par(0)}")
print(f"-2 é par? {e_par(-2)}")
# Utilizando o valor retornado em uma estrutura condicional
numero = 15
if e_par(numero):
print(f"{numero} é um número par.")
else:
print(f"{numero} é um número ímpar.")
```

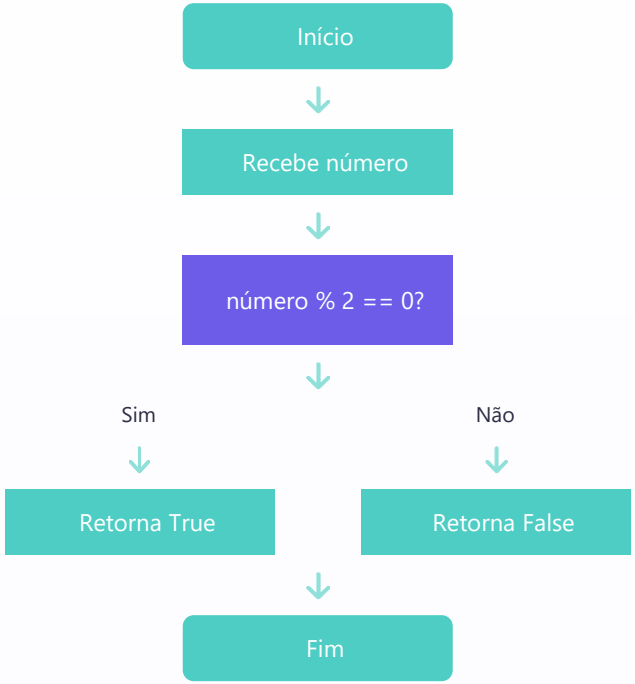
Exercício 8 (prático) (2/2)

```
4 é par? True
7 é par? False
0 é par? True
-2 é par? True
15 é um número ímpar.
```

Casos de Teste

Entrada	Resultado	Explicação
4	True	4 ÷ 2 = 2 com resto 0
7	False	7 ÷ 2 = 3 com resto 1
0	True	0 ÷ 2 = 0 com resto 0
-2	True	-2 ÷ 2 = -1 com resto 0
15	False	15 ÷ 2 = 7 com resto 1

Fluxograma da Função e_par()



Modularização (1/4)

O que é Modularização?

Modularização é a técnica de dividir um programa em partes menores e independentes (módulos), cada uma responsável por uma funcionalidade específica. Em Python, cada ficheiro .py é considerado um módulo.

Ficheiros .py

Um ficheiro .py é um ficheiro de código-fonte Python que pode conter definições de funções, classes e variáveis. Pode ser executado diretamente ou importado como um módulo em outros programas.

Estrutura Típica de um Projeto Python:

- **meu_projeto/**
  - **main.py** *Ponto de entrada principal*
  - **utilidades.py** *Funções auxiliares*
  - **constantes.py** *Valores constantes*
  - **modulos/** *Subpacote com mais módulos*

Exemplo de Módulo

```
# Ficheiro: matematica.py

# Constantes
PI = 3.14159

# Funções
def somar(a, b):
return a + b

def subtrair(a, b):
return a - b

def multiplicar(a, b):
return a * b

def dividir(a, b):
if b == 0:

raise ValueError("Divisão por zero!")

return a / b
```

Modularização (2/4)

Reutilização de Código

A modularização permite reutilizar código em diferentes partes de um programa ou em diferentes programas, evitando duplicação e facilitando a manutenção.

- **Organização:**
  Divide o código em unidades lógicas e funcionais.
- **Manutenção:**
  Facilita a correção de erros e a implementação de melhorias.
- **Reutilização:**
  Permite usar o mesmo código em diferentes projetos.
- **Colaboração:**
  Facilita o trabalho em equipa, com diferentes pessoas trabalhando em diferentes módulos.
- **Testabilidade:**
  Torna mais fácil testar partes isoladas do código.

Nota: O bloco `if __name__ == "__main__":` é uma prática comum em Python. Ele permite que um ficheiro funcione tanto como um script independente quanto como um módulo importável.

Execução de Módulos

Um ficheiro Python pode ser executado diretamente como um script ou importado como um módulo. A variável especial `__name__` ajuda a determinar como o ficheiro está sendo usado.

```python
# Ficheiro: exemplo.py

def funcao_util():
print("Esta função pode ser importada")

# Código que só executa quando o ficheiro é executado diretamente
if __name__ == "__main__":
print("Este ficheiro está sendo executado diretamente")
funcao_util()
```

```
# Quando executado diretamente:
Este ficheiro está sendo executado diretamente
Esta função pode ser importada

# Quando importado como módulo:
(nenhuma saída)
```

Modularização (3/4)

Importação de Módulos

Para usar funções, classes ou variáveis definidas em outros módulos, precisamos importá-los usando a instrução `import` .

```
# Importar um módulo inteiro
import nome_do_modulo

# Importar um item específico de um módulo
from nome_do_modulo import item

# Importar múltiplos itens de um módulo
from nome_do_modulo import item1, item2

# Importar todos os itens de um módulo (não recomendado)
from nome_do_modulo import *
```

Exemplos de Importação

```
# Importar o módulo matemática que criamos
import matematica

# Usar funções do módulo
resultado = matematica.somar(5, 3)
print(f"5 + 3 = {resultado}")

area_circulo = matematica.PI * (2 ** 2)
print(f"Área do círculo: {area_circulo}")
```

```
5 + 3 = 8
Área do círculo: 12.56636
```

```
# Importar funções específicas
from matematica import somar, multiplicar

# Usar as funções diretamente
print(f"5 + 3 = {somar(5, 3)}")
print(f"5 * 3 = {multiplicar(5, 3)}")
```

```
5 + 3 = 8
5 * 3 = 15
```

Modularização (4/4)

Formas de Importação

Sintaxe	Descrição	Uso
import math	Importa o módulo inteiro	math.sqrt(16)
from math import sqrt	Importa função específica	sqrt(16)
import math as m	Importa com alias	m.sqrt(16)
from math import *	Importa tudo (não recomendado)	sqrt(16)

Atenção: Importar tudo com `from modulo import *` não é uma boa prática, pois pode causar conflitos de nomes e dificultar a leitura do código.

Módulos da Biblioteca Padrão

Python vem com uma rica biblioteca padrão que oferece muitos módulos úteis para diversas tarefas.

```
# Módulo random para números aleatórios
import random
print(f"Número aleatório: {random.randint(1, 10)}")

# Módulo datetime para manipulação de datas
from datetime import datetime
agora = datetime.now()
print(f"Data e hora atual: {agora.strftime('%d/%m/%Y %H:%M')}")

# Módulo math para funções matemáticas
import math
print(f"Raiz quadrada de 16: {math.sqrt(16)}")
```

```
Número aleatório: 7
Data e hora atual: 24/06/2025 15:30
Raiz quadrada de 16: 4.0
```

Exercício 9 (prático) (1/3)

Criar um Módulo de Utilidades

Crie um ficheiro `utilidades.py` com funções úteis que já desenvolvemos anteriormente, como `somar` `e_par`, etc. Depois, crie um programa principal que importe e use essas funções.

Requisitos:

1. Criar um ficheiro `utilidades.py` com pelo menos 3 funções úteis
2. Incluir documentação (docstrings) para cada função
3. Criar um programa principal `main.py` que importe o módulo
4. Usar as funções importadas para resolver um problema simples
5. Testar diferentes formas de importação

Estrutura do Projeto:

- **meu_projeto/**
  - **utilidades.py** *Módulo com funções úteis*
  - **main.py** *Programa principal*

```python
# Ficheiro: utilidades.py
def somar(a, b):
"""
Soma dois números e retorna o resultado.

Parâmetros:
a (float): Primeiro número
b (float): Segundo número

Retorna:
float: A soma de a e b
"""
return a + b

def e_par(numero):
"""
Verifica se um número é par.

Parâmetros:
numero (int): O número a ser verificado

Retorna:
bool: True se o número for par, False se for ímpar
```

Continua próximo slide

Exercício 9 (prático) (2/3)

```python
# Ficheiro: utilidades.py (continuação)


    """
    return numero % 2 == 0

def calcular_media(numeros):
    """
    Calcula a média de uma lista de
    números.

    Parâmetros:
    numeros (list): Lista de números

    Retorna:
    float: A média dos números ou 0 se a lista
    estiver vazia
    """
    if not numeros:
    return 0
    return sum(numeros) / len(numeros)
```

```python
# Ficheiro: main.py
# Importar o módulo de utilidades
import utilidades
def main():
print("Demonstração do módulo de
utilidades")
print("-" * 40)
# Usar a função somar
a, b = 10, 5
resultado = utilidades.somar(a, b)
print(f"{a} + {b} = {resultado}")
# Usar a função e_par
for num in [1, 2, 3, 4, 5]:
if utilidades.e_par(num):
print(f"{num} é par")
else:
print(f"{num} é ímpar")
# Usar a função calcular_media
notas = [85, 90, 78, 92, 88]
media =
utilidades.calcular_media(notas)
print(f"A média das notas {notas} é {media:.1f}")
if __name__ == "__main__":
main()
```

Exercício 9 (prático) (3/3)

```
Demonstração do módulo de utilidades
---------------------------------------
10 + 5 = 15
1 é ímpar
2 é par
3 é ímpar
4 é par
5 é ímpar
A média das notas [85, 90, 78, 92, 88] é 86.6
```

Dica:
Este exercício demonstra como organizar o código em módulos reutilizáveis. Ao separar funções em módulos temáticos, você cria uma biblioteca pessoal que pode ser usada em diferentes projetos, economizando tempo e esforço.

Boas Práticas de Programação (1/2)

Indentação

Em Python, a indentação não é apenas uma questão de estilo, mas uma parte fundamental da sintaxe. Ela define blocos de código e afeta a execução do programa.

Incorreto

```
def
calcular_media(notas):
if len(notas) == 0:
return 0
soma = 0
for nota in notas:
soma += nota

return soma / len(notas)
```

Correto

```
def
calcular_media(notas):

if len(notas) == 0:

return 0
soma = 0
for nota in notas:
soma += nota

return soma / len(notas)
```

Nomes Significativos

Use nomes descritivos para variáveis, funções e classes. Nomes bem escolhidos tornam o código mais legível e auto-documentado.

Incorreto

```
def f(x, y):
z = x * y
return z


a = [10, 20, 30, 40,
50]

for i in range(len(a)):

print(f"Item {i}:
{a[i]}")
```

Correto

```
def
calcular_area_retangulo(
largura, altura):

area = largura *
altura

return area


valores = [10, 20, 30,
40, 50]

for indice, valor in
enumerate(valores):

print(f"Item
{indice}: {valor}")
```

Boas Práticas de Programação (2/2)

Comentários

Use comentários para explicar o "porquê" do código, não o "como". Docstrings são essenciais para documentar funções, classes e módulos.

Incorreto

```
# Esta função calcula a
média
def media(lista):

return sum(lista) / len(lis


# Aqui estamos a
verificar se é par
if num % 2 == 0:
print("É par")
```

Correto

```
def
calcular_media(valores):
"""
Calcula a média
aritmética de uma
lista de valores.


Lança ValueError se a
lista estiver vazia.
"""
if not valores:

raise ValueError("Lista
vazia")


return sum(valores) /
len(valores)
```

Outras Boas Práticas

- **DRY (Don't Repeat Yourself):** Evite duplicação de código. Se estiver a repetir o mesmo código, considere criar uma função.
- **KISS (Keep It Simple, Stupid):** Mantenha o código simples e direto. Soluções complexas são mais difíceis de entender e manter.
- **Funções pequenas e focadas:** Cada função deve fazer apenas uma coisa, mas fazê-la bem.
- **Tratamento de erros:** Use try/except para lidar com exceções e validar entradas.
- **Consistência:** Siga um estilo de codificação consistente em todo o projeto.
- **PEP 8:** Siga as convenções de estilo do Python (PEP 8) para melhorar a legibilidade.

```
# Exemplo de tratamento de erros
def dividir(a, b):
try:
return a / b
except ZeroDivisionError:
print("Erro: Divisão por zero!")
return None
```

Aplicações Práticas (1/2)

Gerador de Passwords

Um script simples para gerar passwords aleatórias com diferentes níveis de complexidade.

```
import random
import string

def gerar_password(comprimento=12,
    incluir_maiusculas=True,
    incluir_numeros=True,
    incluir_especiais=True):

    caracteres = string.ascii_lowercase
    if incluir_maiusculas:
        caracteres += string.ascii_uppercase
    if incluir_numeros:
        caracteres += string.digits
    if incluir_especiais:
        caracteres += string.punctuation
    return ''.join(random.choice(caracteres)
        for _ in range(comprimento))

# Exemplos de uso
print("Password:", gerar_password())
print("Simples:", gerar_password(8,False,False,False))
```

```
Password: Kj7$mP9!xR2@
Simples: qwertyzx
```

Calculadora de Estatísticas

Script para calcular estatísticas básicas (média, mediana, moda) de uma lista de valores.

```
def calcular_estatisticas(valores):
    if not valores:
        return None
    valores_ord = sorted(valores)
    n = len(valores)

    media = sum(valores) / n

    if n % 2 == 0:
        mediana = (valores_ord[n//2-1]
            + valores_ord[n//2]) / 2
    else:
        mediana = valores_ord[n//2]

    minimo = min(valores)
    maximo = max(valores)
    return {"média": media,
            "mediana": mediana,
            "mínimo": minimo,
            "máximo": maximo}

notas = [85, 90, 78, 92, 88, 76, 95, 89]
```

```
média: 86.6
mediana: 88.5
mínimo: 76.0  |  máximo: 95.0
```

Aplicações Práticas (2/2)

Processamento de Texto

Script para processar texto: contar palavras, caracteres e encontrar padrões.

```
def analisar_texto(texto):
    if not texto:
        return {}

    linhas = texto.split('\n')
    palavras = texto.split()
    caracteres = sum(len(p) for p in palavras)
    palavra_longa = max(palavras, key=len)

    return {
        "num_linhas": len(linhas),
        "num_palavras": len(palavras),
        "num_caracteres": caracteres,
        "palavra_mais_longa": palavra_longa
    }

# Exemplo de uso
texto = """Python é uma linguagem
de alto nível, interpretada."""
stats = analisar_texto(texto)
for k, v in stats.items():
    print(f"{k}: {v}")
```

```
num_linhas: 4
num_palavras: 19
num_caracteres: 119
palavra_mais_longa: programação
```

Contagem de Linhas em Ficheiros

```
def contar_linhas(caminho):
    with open(caminho, 'r') as f:
        linhas = f.readlines()
    total = len(linhas)
    branco = sum(1 for l in linhas
        if l.strip() == '')
    comentario = sum(1 for l in linhas
        if l.strip().startswith('#'))
    codigo = total - branco - comentario
    return {"total": total,
            "codigo": codigo,
```

```
total: 42 | codigo: 25 | comentario: 8
```

Análise de Dados Simples (CSV)

```
def analisar_csv(caminho, sep=','):
    with open(caminho, 'r') as f:
        linhas = f.readlines()
    cabecalhos = linhas[0].strip().split(sep)
    dados = [l.strip().split(sep)
        for l in linhas[1:]]
    print(f"Ficheiro: {caminho}")
    print(f"Registos: {len(dados)}")
    print(f"Campos: {', '.join(cabecalhos)}")
    return {"cabecalhos": cabecalhos,
            "num_registos": len(dados),
```

```
Registos: 100 | Campos: Nome, Idade, Cidade
```

Desafio Final (Mini-Projeto) (1/2)

Sistema de Gestão de Contactos

Desenvolva um programa completo para gerir uma lista de contactos, permitindo adicionar, visualizar, pesquisar e remover contactos.

Requisitos:

1. Criar uma estrutura modular com pelo menos 3 ficheiros Python
2. Implementar um menu interativo com opções para todas as funcionalidades
3. Armazenar contactos em uma lista de dicionários com nome, telefone e email
4. Validar todas as entradas do utilizador (telefone, email, etc.)
5. Implementar funções de pesquisa por nome ou telefone
6. Permitir guardar e carregar contactos de um ficheiro

Estrutura do Projeto:

gestor_contactos/

- **main.py** *Programa principal com menu*
- **contactos.py** *Funções para gerir contactos*
- **utils.py** *Funções auxiliares e validações*
- **contactos.json** *Ficheiro de dados*

Conceitos a Aplicar

- Estruturas de dados (listas, dicionários)
- Funções e modularização
- Estruturas de controlo (condicionais, ciclos)
- Entrada e saída de dados
- Manipulação de ficheiros
- Validação de dados
- Tratamento de exceções

Desafio Final (Mini-Projeto) (2/2)

Exemplo de Implementação

```python
# contactos.py
def adicionar_contacto(lista_contactos, nome, telefone, email):
"""Adiciona um novo contacto à lista."""
novo_contacto = {
"nome": nome,
"telefone": telefone,
"email": email
}
lista_contactos.append(novo_contacto)
return True

def pesquisar_por_nome(lista_contactos, nome):
"""Pesquisa contactos pelo nome."""
resultados = []
for contacto in lista_contactos:
if nome.lower() in contacto["nome"].lower():
resultados.append(contacto)
return resultados
```

```python
# main.py (trecho)
def exibir_menu():
print("\n===== GESTOR DE CONTACTOS =====")
print("1. Adicionar contacto")
print("2. Listar todos os contactos")
print("3. Pesquisar contacto")
print("4. Remover contacto")
print("5. Guardar contactos")
print("6. Carregar contactos")
print("0. Sair")
return input("Escolha uma opção: ")
```

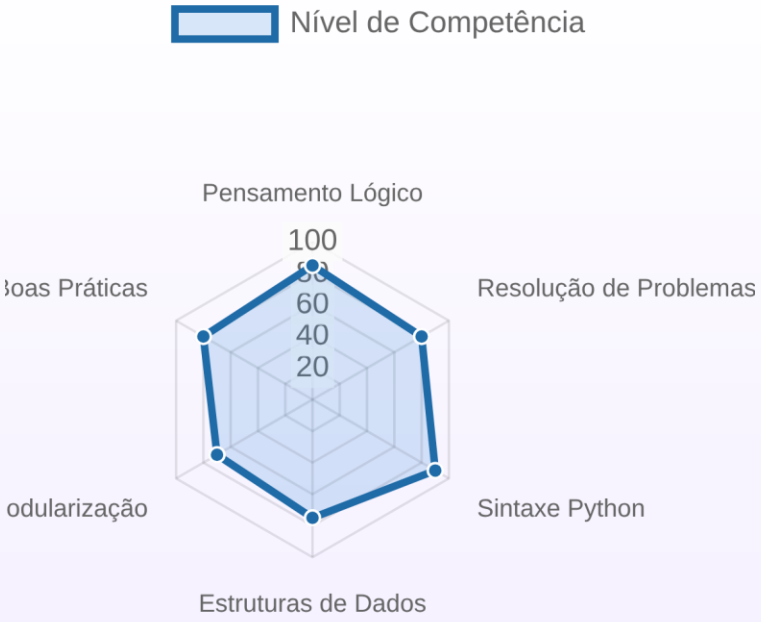
Dica:
Este projeto integra todos os conceitos aprendidos durante o curso. Comece com uma versão simples e vá adicionando funcionalidades gradualmente. Teste cada função separadamente antes de integrá-la ao programa principal.

Resumo Final da UC (1/2)

Objetivos Alcançados

- Compreensão dos conceitos fundamentais da programação estruturada
- Domínio da sintaxe básica da linguagem Python
- Capacidade de utilizar diferentes tipos de dados e estruturas de controlo
- Desenvolvimento de algoritmos para resolver problemas simples e complexos
- Aplicação de boas práticas de programação e modularização de código

Competências Adquiridas



Resumo Final da UC (2/2)

Jornada de Aprendizagem

1. Conceitos Básicos
2. Estruturas de Controlo
3. Estruturas de Dados
4. Funções
5. Modularização

Aplicações Práticas

- Desenvolvimento de scripts para automação de tarefas
- Processamento e análise de dados
- Manipulação de ficheiros e texto
- Criação de aplicações com interface de linha de comando

Próximos Passos

- Programação orientada a objetos
- Desenvolvimento de interfaces gráficas
- Integração com bases de dados
- Desenvolvimento web com frameworks Python

> *"A programação é uma habilidade que, uma vez adquirida, abre portas para um mundo de possibilidades. O que aprendemos aqui é apenas o começo de uma jornada fascinante."*
>
> ***— Mário Amorim***

Contactos e Encerramento

Programação Estruturada e Tipos de Dados

Contactos do Docente

Prof. Mário Amorim

mario.amorim@my.istec.pt

Horário de Atendimento

Dia	Horário
Terça-feira	13:00 - 14:00
Quarta-feira	13:00 - 14:00
Sexta-feira	13:00 - 14:00

Obrigado pela vossa atenção!

Espero que tenham gostado desta unidade curricular e que continuem a explorar o fascinante mundo da programação. Estou disponível para esclarecer dúvidas e ajudar no vosso percurso de aprendizagem.